

Helix OTA — Future Phase: Linux Support (all flavors)

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	future (research outline)
Status summary	Basic planning + research for extending Helix OTA to Linux distributions of all types/flavors, via the OS-adapter seam. The control plane stays unchanged; a Linux device adapter is added. Grounded in the already-completed research notes for RAUC, OSTree/bootc, and SWUpdate.
Issues	Multiple coexisting Linux update standards (image-based A/B vs file-based atomic vs package-manager) — Helix must support all relevant ones via pluggable backends.
Issues summary	No single Linux mechanism fits all; the adapter must select per target class.
Fixed	initial future outline
Fixed summary	Captured the candidate engines + decision axes per the operator “support it all” mandate.
Continuation	Promote to a numbered phase (e.g. 2.0.0-linux) with full specs + a hands-on spike per backend when Android phases are complete.

Table of contents

- [§1. Scope](#)
- [§2. Candidate backends \(researched\)](#)
- [§3. OS-adapter contract](#)
- [§4. Per-class strategy](#)
- [§5. Open spikes](#)

§1. Scope

Embedded Linux, server Linux, and desktop Linux across distros (Debian/Ubuntu, Fedora/RHEL, Arch, SUSE, Yocto/buildroot images). The Helix control plane (artifact intake, signing/verify, staged rollout, telemetry) is reused unchanged; only a Linux **device adapter** implementing the `ota-protocol` + OS-adapter interface is added.

§2. Candidate backends (researched)

See the decision-grade notes already produced:

- **RAUC** (`research/stacks/rauc.md`) — leading candidate: fail-safe A/B bundles, X.509/CMS verification (exceeds ad-hoc signing), HTTP(S) streaming that fits the Go control plane, D-Bus API, broad bootloader support. Cost: per-target bootloader integration + a self-maintained D-Bus binding.
- **OSTree / libostree / bootc** (`research/stacks/ostree.md`) — file-based atomic deployments with pinning + static deltas; archive-over-HTTP server drops into Go + S3/MinIO; strong for desktop/immutable distros. Evaluate `bootc` given the ecosystem shift.
- **SWUpdate** (`research/stacks/swupdate.md`) — embedded Linux `.swu`, Lua handlers, native hawkBit (suricata) integration; relevant for constrained embedded targets.

§3. OS-adapter contract

The universal seam (master design §4): `CheckForUpdate` / `Download(progress)` / `Verify(hashes+signature)` / `Install` / `Rollback` / `GetCapabilities`. Each Linux backend (RAUC, OSTree, SWUpdate, package-manager) is a concrete adapter behind this interface, selected by target class. This is the same seam the Android `update_engine` bridge implements — proving the decoupling.

§4. Per-class strategy

Target class	Backend	Rationale
Embedded A/B device	RAUC	fail-safe A/B + bootloader integration
Immutable/desktop	OSTree / bootc	atomic file-based + pinning rollback
Constrained embedded	SWUpdate	small footprint, Lua flexibility
Mutable server/distro	package-manager adapter (APT/DNF/Pacman) + snapshot	where image-based isn't viable

§5. Open spikes

Per backend: build a signed bundle/commit → stream from the Go control plane over HTTP Range → drive the install/commit/mark API → confirm rollback. Measure delta sizes + storage. Confirm the UNVERIFIED items flagged in each stack note against pinned releases before committing an ADR.

§6. addition_3_research_routing (rev 2 — 2026-06-08)

This section folds addition-#3's Linux research into this canonical phase dir per synthesis §11 (gap §11 future-phase catalogue). It **extends** the original outline above; nothing above is removed. Numbering note: the source doc self-numbered **1.1.0-linux-support**; the canonical destination is **1.X-linux** (promoted to a concrete version when Android phases 1.0.1–1.0.3 are complete), per synthesis §11's 1.X-linux/ routing.

§6.1 source_research

Source: additions/initial_research_03/helix_ota_big/1.1.0-linux-support/docs/LINUX_OTA_RESEARCH.md. Cited sections, summarized (NOT copied):

- **§1 Linux OTA Landscape** — package-based vs image-based/atomic vs hybrid; comparison with Android's A/B model (§1.5). Confirms the per-class split already captured in §4 above.
- **§2 Ubuntu/Debian** — APT, unattended-upgrades, A/B rootfs (Ubuntu-Core style), snap trade-offs, custom APT repo for OTA, and an OSAdapter-interface implementation plan (§2.7).
- **§3 Fedora/rpm-ostree** — atomic update mechanism, OSTree repo hosting, static-delta generation/serving (§3.3), and **bootc / bootable containers** as the forward direction (§3.4) — matches §2's "evaluate bootc" note above.
- **§4 Arch** — pacman + Btrfs A/B snapshot approach (§4.2), delta packages.
- **§5 Generic Linux A/B rootfs** — distro-agnostic A/B partition scheme with **GRUB** (§5.2) and **U-Boot** (§5.3) slot-switch + 3-strike boot-retry integration, and an ABRootfsManager (§5.4).
- **§6 OSAdapter plugin interface** — Go interface definition (§6.1), plugin loading (§6.2), per-distro adapters, config schema, per-adapter testing — the concrete form of the universality seam in §3 above.
- **§7 New submodules** — helix-linux-client daemon, helix-ostree-adapter, helix-apt-adapter, helix-pacman-adapter.
- **§8 Migration path** — extend the existing server for multi-OS (§8.1), dashboard changes (§8.2), API versioning (§8.3), migration checklist (§8.4).

§6.2 reconciliation_to_locked_decisions

- **Control plane unchanged:** Go + **Gin**, REST primary (HTTP/3→HTTP/2, Brotli); Linux support is device-adapter-only. The source's "extend the server" framing (§8.1) is accepted only as *adapter wiring*, not control-plane forking.

- **Vocabulary:** canonical **releases + deployments**; the source's updates/rollouts terms are re-based.
- **Signing/trust:** **ed25519 + SHA-256** at the Helix layer; RAUC's X.509/CMS and OSTree GPG/sig verification are *additional* platform-native layers, not replacements. JWT bearer for device auth (mTLS optional hardening).
- **Module path / submodules:** re-base `helix-*`-adapter names to the `ota-*` convention under `github.com/HelixDevelopment`; gate PUBLIC repo creation on the same G11 verification as all `ota-*` repos. Reuse the verified catalogue, not invented modules.
- **Coverage:** $\geq 90\%$ floor on safety-critical adapter paths (verify, install, rollback/commit).

§6.3 what_must_be_specced_before_this_phase_starts

1. Pick the per-class backends via an ADR (RAUC / OSTree-bootc / SWUpdate / package-manager+snapshot) — resolve the §4 table against pinned-release evidence.
2. Freeze the **OSAdapter Go interface** (the same seam Android `update_engine` and Windows implement) before writing any adapter.
3. Spike each backend end-to-end (build signed artifact → stream from Go control plane over HTTP Range → drive install/commit/mark → confirm rollback) and **measure** delta/storage.
4. Confirm a reliable **post-install health signal** per backend (for the health-gated halt that 1.0.1 introduces).
5. Re-based submodule specs (`ota-linux-client`, `ota-ostree-adapter`, `ota-apt-adapter`, `ota-pacman-adapter`) + PUBLIC repo verification.
6. Promote `1.X-linux` to a concrete version number once Android 1.0.1–1.0.3 land.

§6.4 anti_bluff_unverified_register

Per Constitution §11.4.6 / §7.1 — MUST NOT propagate as fact:

- All RAUC/OSTree/SWUpdate/bootc capability claims and the source's per-distro feasibility assertions — UNVERIFIED until spiked against pinned releases (carried from the `research/stacks/*` notes).
- GRUB / U-Boot 3-strike boot-retry behavior on real Linux targets — UNVERIFIED hardware gate.
- Static-delta sizes and storage numbers — UNVERIFIED; measure.
- Source-doc Go OSAdapter reference + `helix-*` submodule names — reference only; non-canonical until re-based.
- All cited HelixConstitution §11.4.x clause numbers remain UNVERIFIED except the six confirmed in `tests/test_strategy.md` §13.